第 13 章練習解答

1

取 edges $s \to a$ 權重 2、$s \to b$ 權重 1、$a \to b$ 權重 -2。Dijkstra 先固定 $d(b) = 1$，再固定 $d(a) = 2$；之後從 a 鬆弛 b 得到真正距離 0，但標準 Dijkstra 已不再更新 b，所以會留下錯誤值。這張圖沒有 negative cycle。它證明不能省略通用正確性定理的非負權重前提，不表示每張含負邊的圖都必然算錯。

2

把 edges 依權重排序為 $AB : 1$、$BC : 2$、$CD : 3$、$AC : 4$、$BD : 5$。Kruskal 依序取 AB, BC, CD，總權重 $1 + 2 + 3 = 6$。若取 cut $S = \{A\}$，crossing edges 是 AB 與 AC，其中 AB 權重最小。最後的 cut $S = \{A, B, C\}, T = \{D\}$ 有 CD:3 與 BD:5 兩條 crossing edges，CD 並不是唯一跨邊，但它是唯一最輕跨邊，因此必在每棵 MST 中。這個 cut 尊重已選 AB、BC，安全邊證明保證加入 CD 後仍可擴成 MST；此時已是 spanning tree。

3

例如 L_1, L_2 都只連到 R_1，L_3 連到 R_2。對 $S = \{L_1, L_2\}$，neighborhood $N(S) = \{R_1\}$，於是

$$|N(S)| = 1 < |S| = 2,$$

所以 Hall 條件失敗。最大 matching 可取 $L_1 \to R_1, L_3 \to R_2$；也可把 L_1 換成 L_2，兩種 matching 的大小都是 2。

4

unit capacities 使每條 $source \to L$ 與 $R \to sink$ arc 的 flow 至多為 1。由 flow conservation，若某個 L 有 incoming flow 1，其 outgoing flow 總和也必須為 1。題目指定 integral maximum flow，因此每條 arc 的 flow 為 0 或 1。整數 capacities 保證存在 integral maximum flow，例如從零流開始反覆做整數 augmenting path；這不表示任意 feasible flow 都必然為整數。若某個 R 有 outgoing flow 1，其 incoming flow 總和也只能為 1，因此每個 L 與 R 最多各接一條 used edge。這些選中的 $L \to R$ edges 不共享 endpoint，正是 bipartite matching。

5

原始 capacities 為 $s \to a : 3$、$s \to b : 2$、$a \to b : 1$、$a \to t : 2$、$b \to t : 3$。先送 $s \to a \to t$ 的 flow 2，再送 $s \to b \to t$ 的 flow 2，共得 flow 4。殘餘容量允許再送 $s \to a \to b \to t$ 的 flow 1，此時 source 的兩條 outgoing arcs 分別飽和為 3 與 2，總 flow 為 5。

第 13 章練習解答 （續）

Cut $S = \{s\}$ 的容量是 $3 + 2 = 5$，而任何 flow 都不超過 cut capacity。現有 flow 已達 5，所以 maximum flow 為 5，且此 cut 是最小 cut。終態從 s 已沒有正向 residual capacity，residual graph 中從 s 可達的集合只有 $\{s\}$。

6

只依權重加入 edges 而不檢查 cycle，可能形成 cycle；若無條件加入所有排序 edges，結果也不必然是 forest。正確的 Kruskal 用 disjoint-set union（DSU）維護 components：只有 `find(u) != find(v)` 時才 union，並把這條 edge 加入 minimum spanning forest。若圖連通，接受 edges 到 $n - 1$ 條即停止；若圖不連通，則對每個 component 各產生一棵 spanning tree。

7

初始化 d=(0,∞,∞,∞,∞,∞)，依 A、B、C、D、E、F 排列。確定 A 後為 (0,3,1,∞,∞,∞)，B←A、C←A；確定 C 後為 (0,2,1,8,∞,∞)，B←C、D←C；確定 B 後為 (0,2,1,4,8,∞)，D←B、E←B；確定 D 後為 (0,2,1,4,4,∞)，E←D；確定 E 後不變。已確定集合依序 {A}、{A,C}、{A,C,B}、{A,C,B,D}、{A,C,B,D,E}。剩 F 為 ∞，停止。

到 E 的 path 為 A–C–B–D–E，成本 1+1+2+0=4。逐邊檢查：

$$d_B = 2 \le 0 + 3,\ d_C = 1 = 0 + 1,\ d_B = 2 = 1 + 1,$$
$$d_D = 4 = 2 + 2,\ d_D = 4 \le 1 + 7,\ d_E = 4 = 4 + 0,\ d_E = 4 \le 2 + 6.$$

每個有限 d 都由 predecessor path 達到；F 不可由 A 可達。新增只在 F 上的負權自環，不改 A 的可達區域，故以上距離仍成立。若實作選擇更保守的「全圖邊非負」輸入檢查，它可能拒絕輸入；這是實作前提更嚴格，不表示數學上此負環會影響 A 的答案。

8

若某非樹邊 e 的權重小於其樹路徑最大邊 f，把 e 加入再刪 f 可嚴格降低成本，所以 T 根本不是 MST。若相等，同樣交換得另一棵不同的同重生成樹，所以 T 不唯一。因此唯一 MST 必有全部嚴格大於。

反向假設每條非樹邊 e 都嚴格大於其 T-path 上所有邊。e 加上該 path 構成 cycle，且 e 是唯一最重邊，由 cycle property，任何 MST 都不能包含 e。所有非樹邊都被排除，因此任何 MST 的 n−1 條邊只能是 T 的 n−1 條邊，唯一 MST 就是 T。

四個環邊中任刪一條都形成生成樹，成本 3。任何含對角線的生成樹至少成本 2+1+1=4，故全部 MST 正是

$$\{AB, BC, CD\}, \{BC, CD, DA\}, \{CD, DA, AB\}, \{DA, AB, BC\}.$$

第 13 章練習解答 （續）

共四棵，沒有一條邊出現在每棵 MST：每個環邊都在刪掉自己時缺席；兩條對角邊在全部 MST 都缺席。最小總重相同不代表樹相同。

9

以原始弧順序 (sa,sb,ab,at,bt,ba) 記流量。第一次後為 (1,0,1,0,1,0)。方向 b→a 有兩個 residual records：原始 ba 的正向 record，容量 2−0=2；原始 ab 的反向 record，容量 f(ab)=1。若只為搜尋合併，容量為 3，但不能把這個 3 當成單一原始弧的剩餘容量。

選原始 ba 的正向 record，下一次增加 sb、ba、at 各 1，得到 (1,1,1,1,1,1)。a、b 中各有一單位的 ab、ba 循環，總流仍為 2。選 ab 的反向 record 則增加 sb、at 各 1、減 ab 一單位，得到 (1,1,0,1,1,0)，沒有這個循環。兩者都逐弧在容量內，a、b 都守恆，且割 {s} 容量 2，故兩者都是最大流。最大流值唯一，不表示每條原弧的最優流量唯一。

10

取 M={a1,b2,d3}，大小 3。從未匹配左點 c 搜尋：c 經非匹配邊到 1、2，再經匹配邊到 a、b；a、b 的其他鄰居仍只有 1、2。故 Z_L={a,b,c}、Z_R={1,2}，沒有未匹配右端可增廣。S=Z_L 的 |S|−|N(S)|=3−2=1，證任何匹配至多 4−1=3，與 M 相等。

Kőnig 構造 C=(L−Z_L)∪Z_R={d,1,2}，確實覆蓋所有邊，大小 3，因此是 minimum cover。{1,2,3} 也可行且同大小，最小 cover 不必唯一。

增加 c4 後，{a1,b2,c4,d3} 為四邊 perfect matching。任何 matching 最多四邊，已達上限。沒有未匹配左點，所以交錯搜尋集合全空，構造 cover=L，大小 4，並由四條互不共端的 matching edges 得到同值下界。

11

必要性由 S 的全部需求只能流向 N(S)，其接收總容量是 b(N(S))，直接得到 d(S)≤b(N(S))。

充分性令 D=$\Sigma_i d_i$，建 s→i 容量 d_i、i→j 容量 D+1（僅在原 pair 有邊時）、j→t 容量 b_j。割 {s} 容量 D，故最小割至多 D。任何跨越某條 left→right 弧的割容量至少 D+1，不可能是較小割。其餘割若 source 側左點為 S，source 側右點必包含 N(S)，容量至少

$$D - d(S) + b(N(S)) \geq D.$$

因此每個割容量至少 D，最大流最小割定理給最大流值 D。容量全整數，可取整數最大流；全部 s→i 都被飽和，便交付各左點全部需求。D=0 時零流也符合此構造。

第 13 章練習解答 （續）

案例以 A、B 表示左點、1、2 表示右點，分配 A→2 為 2 單位，B→1 為 1 單位，滿足需求及容量。若每個 pair 最多一單位，此分配不可用，而且仍無任何替代可行解：B 必用 1，使 1 的容量已滿；A 只能用 2，但 pair A2 至多 1，不能滿足需求 2。原所有 neighborhood 總容量條件仍成立，所以此版本不再充分；必須在 flow network 的 pair arcs 加上 1 的容量，再以完整 cut 條件處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 13 章練習解答 （續）

原創驗算程式附錄: residual_network.py

```
"""Edmonds–Karp with separate residual pairs for every original arc ID."""
from collections import deque

def maximum_flow(n: int, arcs: list[tuple[int,int,int]], source: int, sink: int):
    if not 0 <= source < n or not 0 <= sink < n or source == sink:
        raise ValueError("source and sink must be different valid vertices")
    graph=[[] for _ in range(n)]
    originals=[]
    for u,v,capacity in arcs:
        if not 0 <= u < n or not 0 <= v < n or not isinstance(capacity,int) or capacity < 0:
            raise ValueError("arc endpoints must be valid and capacity a nonnegative
integer")
        at_u=len(graph[u]);at_v=len(graph[v])+(u==v)
        graph[u].append([v,at_v,capacity])
        graph[v].append([u,at_u,0])
        originals.append((u,at_u,capacity))
    value=0
    while True:
        parent=[None]*n;parent[source]=(source,-1);queue=deque([source])
        while queue and parent[sink] is None:
            u=queue.popleft()
            for i,(v,reverse,residual) in enumerate(graph[u]):
                if residual>0 and parent[v] is None:
                    parent[v]=(u,i);queue.append(v)
        if parent[sink] is None:
            flows=[capacity-graph[u][i][2] for u,i,capacity in originals]
            reached={v for v in range(n) if parent[v] is not None}
            return value,flows,reached
        amount=None;v=sink
        while v!=source:
            u,i=parent[v]
            amount=graph[u][i][2] if amount is None else min(amount,graph[u][i][2])
            v=u
        v=sink
        while v!=source:
            u,i=parent[v];record=graph[u][i]
            record[2]-=amount
            graph[v][record[1]][2]+=amount
            v=u
        value+=amount

if __name__=='__main__':
    arcs=[(0,1,1),(0,2,1),(1,2,1),(1,3,1),(2,3,1),(2,1,2)]
    value,flows,source_side=maximum_flow(4,arcs,0,3)
    assert value==2 and source_side=={0}
    assert all(0<=flow<=arc[2] for flow,arc in zip(flows,arcs))

print({'value':value,'original_arc_flows':flows,'minimum_cut_source_side':sorted(source_side)
})
```

第 13 章練習解答 （續）

原創驗算程式附錄: verify_graphs.py

```
"""Check the chapter's shortest-path, MST, and max-flow examples."""

import heapq


def dijkstra(n: int, edges: list[tuple[int, int, int]], source: int, directed: bool = True) -
> list[float]:
    if not 0 <= source < n:
        raise ValueError("source must be a vertex")
    if any(w < 0 or not 0 <= u < n or not 0 <= v < n for u, v, w in edges):
        raise ValueError("this implementation requires all graph edges to have nonnegative
weights")
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
        if not directed:
            graph[v].append((u, w))
    dist = [float("inf")] * n
    dist[source] = 0
    queue = [(0, source)]
    while queue:
        d, u = heapq.heappop(queue)
        if d != dist[u]:
            continue
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(queue, (nd, v))
    return dist


def mst(n: int, edges: list[tuple[int, int, int]]) -> int:
    parent = list(range(n))

    def find(x: int) -> int:
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    total = used = 0
    for w, u, v in sorted((w, u, v) for u, v, w in edges):
        ru, rv = find(u), find(v)
        if ru != rv:
            parent[ru] = rv
            total += w
            used += 1
    assert used == n - 1
    return total
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 13 章練習解答 （續）

```
def max_flow(capacity: list[list[int]], source: int, sink: int) -> int:
    n = len(capacity)
    if source == sink or not 0 <= source < n or not 0 <= sink < n:
        raise ValueError("source and sink must be different vertices")
    if any(len(row) != n or any(c < 0 for c in row) for row in capacity):
        raise ValueError("capacity must be a square nonnegative matrix")
    residual = [row[:] for row in capacity]
    flow = 0
    while True:
        parent = [-1] * n
        parent[source] = source
        queue = [source]
        for u in queue:
            for v, cap in enumerate(residual[u]):
                if cap > 0 and parent[v] == -1:
                    parent[v] = u
                    queue.append(v)
                    if v == sink:
                        break
        if parent[sink] == -1:
            return flow
        add = float("inf")
        v = sink
        while v != source:
            u = parent[v]
            add = min(add, residual[u][v])
            v = u
        v = sink
        while v != source:
            u = parent[v]
            residual[u][v] -= add
            residual[v][u] += add
            v = u
        flow += add


assert dijkstra(2, [(0, 1, 2)], 0) == [0, 2]
assert dijkstra(2, [(0, 1, 2)], 1) == [float("inf"), 0]
assert dijkstra(2, [(0, 1, 2_000_000_000)], 0) == [0, 2_000_000_000]
assert mst(4, [(0, 1, 1), (1, 2, 2), (2, 3, 3), (0, 2, 4), (1, 3, 5)]) == 6

cap = [[0] * 4 for _ in range(4)]
cap[0][1], cap[0][2], cap[1][2], cap[1][3], cap[2][3] = 3, 2, 1, 2, 3
assert max_flow(cap, 0, 3) == 5

print("graph examples: PASS")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study